Smart Job Scheduling System

Job Sequencing with Deadlines using a Greedy Algorithm

Course Details: Design and Analysis of Algorithms (202045601)
Presented by: Mayuri Dudhat – 12402080601078
Semester : V
Branch : IT

Problem Statement & Objective

The Problem

- Given n jobs, each with an ID, name, deadline, profit and priority.
- Every job takes one unit of time and only one job can run per time slot.
- A job earns its profit only if it finishes on or before its deadline.

Objective

- Select and order jobs to maximize total profit.
- Use priority to break ties between equal-profit jobs.
- Report schedule, total profit, slot utilization and unscheduled jobs.

Greedy Approach

1	**Sort**	Order jobs by profit (high to low); ties broken by higher priority.
2	**Find slots**	Find the maximum deadline to know how many time slots exist.
3	**Place**	For each job, scan slots from its deadline down to slot 1.
4	**Assign**	Take the first free slot; if none is free, the job is skipped.
5	**Report**	Print schedule, profit, utilization and unscheduled jobs.

Implementation (C++)

Data Structure

```
struct Job {
  id, name,
  deadline, profit,
  priority,
  scheduled
}
```

Array of up to 20 jobs; string schedule[100] holds the time slots.

Key Functions

- sortJobs(): bubble sort by profit, then priority
- scheduleJobs(): greedy slot assignment and statistics
- displayJobs(): formatted job table

Menu-Driven Flow

- Input n (1 to 20) and job details
- 1. Display Jobs
- 2. Generate Schedule
- 3. Exit

Worked Example

Job	Deadline	Profit	Result
J1	2	100	Slot 2
J3	2	27	Slot 1
J4	1	25	Skipped
J2	1	19	Skipped
J5	3	15	Slot 3

Jobs shown in sorted order (by profit)

Output

Slot 1: J3
Slot 2: J1
Slot 3: J5

Scheduled: 3 of 5
Total profit: 142
Utilization: 100%

Complexity Analysis

$O(n^2)$	Sorting (bubble sort)
$O(d)$	Slot search per job
$O(n \cdot d)$	Scheduling (all jobs)
$O(d)$	Space (schedule array)

Overall: $O(n^2)$ time, $O(d)$ space
(d = maximum deadline). Greedy is optimal here: picking the most profitable job first and giving it the latest free slot never blocks a better choice. Sorting with merge sort and slot lookup with Union-Find would bring it to O(n log n).

Conclusion & Future Scope

Conclusion

- Greedy strategy maximizes profit while respecting deadlines.
- Priority tie-breaking gives deterministic, sensible schedules.
- Utilization and unscheduled-job reports make results easy to read.

Future Scope

- Faster sorting and Union-Find for O(n log n).
- Dynamic arrays instead of fixed limits (20 jobs, 100 slots).
- Allow job names with spaces and input validation.
- Support jobs with different durations.